LAPORAN PRAKTIKUM

PENGOLAHAN CITRA DAN VISI KOMPUTER

MODUL 9

MORFOLOGI: EROSI, DILASI, OPENING, CLOSING, TRANSFORMASI TOP-HAT DAN BOTTOM-HAT, SKELETON, THICKENING

Disusun oleh:

Afrizal Rafli Kusuma Wardana

NIM 244107020007

Kelas TI-3C

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

D. TUGAS PRAKTIKUM

Github: https://github.com/rizalrfli/PCVK_GANJIL-2026/blob/main/WEEK_9/Week9.ipynb

Persiapan Lingkungan Kerja

Praktikum dikerjakan pada Google Colaboratory. Repository GitHub praktikum minggu sebelumnya dibuka, lalu berkas notebook di-rename menjadi Week9.ipynb. Folder citra pada Google Drive dihubungkan dengan perintah berikut.

```
from google.colab import drive
drive.mount('/content/drive')
```

Selanjutnya beberapa library yang dipakai selama praktikum diimpor, yaitu OpenCV, NumPy, dan Matplotlib. Untuk langkah skeleton dan thickening ditambahkan scikit-image.

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
```

Agar notebook dapat dijalankan baik di Colab maupun di komputer lokal, proses mount Drive dibungkus blok try-except dan lokasi folder citra dipilih secara otomatis. Berkas uji yang dipakai pada laporan ini disimpan di folder Images pada repository: plat nomer.jpg berukuran 640 x 260 piksel, j.png berukuran 320 x 320 piksel, kitten01.jpg berukuran 520 x 346 piksel, lily.jpg berukuran 640 x 426 piksel, dan ktp.jpg berukuran 900 x 600 piksel. Nama berkas dibuat sama dengan modul, sehingga saat dijalankan di Colab dengan citra dari Drive, seluruh sel berjalan tanpa perubahan kode.

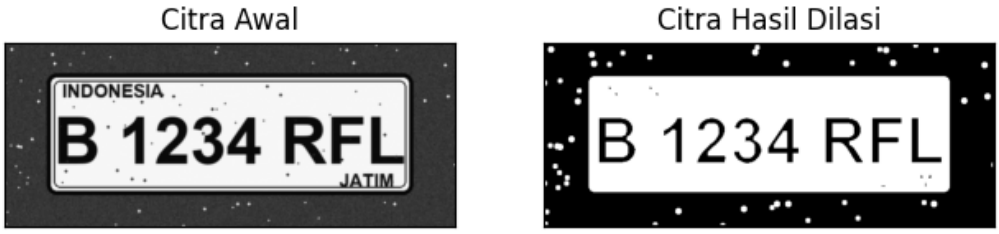
Langkah 3. Dilasi tanpa Library OpenCV

Dilasi dihitung manual dengan menggeser SE berukuran w x w pada seluruh piksel citra, mengalikan jendela citra dengan SE, lalu mengambil nilai maksimum dari hasil perkalian. Nilai maksimum dipakai karena dilasi hanya membutuhkan satu piksel objek di dalam jendela agar piksel pusat menjadi objek.

```
def dilasi_citra(F, w):
    p, q = F.shape
```

```
    imgD = np.zeros((p, q), dtype=np.uint8)
    SED = np.ones((w, w), dtype=np.uint8)      # structuring element persegi
    constant1 = (w - 1) // 2
    for i in range(constant1, p - constant1):
        for j in range(constant1, q - constant1):
            temp = F[i-constant1:i+constant1+1, j-constant1:j+constant1+1]
            product = temp * SED
            imgD[i, j] = np.max(product)
    return imgD

img_d = cv2.imread(path('plat nomer.jpg'), 0)
ret, thresh = cv2.threshold(img_d, 125, 200, cv2.THRESH_BINARY)
w = 5
imgD = dilasi_citra(thresh, w)
```



Gambar 1. Hasil dilasi manual dengan SE persegi 5 x 5

Jumlah piksel objek bertambah dari 63.367 menjadi 77.866 piksel, atau naik 14.499 piksel. Pertambahan terjadi di tepi objek, bukan di tengah objek, sehingga bentuk dasar plat nomor tetap dipertahankan. SE persegi 5 x 5 menambah ketebalan ke segala arah termasuk diagonal, akibatnya huruf tampak lebih tebal dan bintik putih kecil di latar ikut membesar.

Judul langkah pada modul menyebut SE berbentuk cross 3 x 3, sedangkan potongan kode modul membangkitkan SE persegi dengan np.ones. Kedua versi dijalankan. SE cross 3 x 3 didefinisikan eksplisit sebagai matriks berikut.

```
SED = np.array([[0, 1, 0],
                [1, 1, 1],
                [0, 1, 0]], dtype=np.uint8)
imgD3 = dilasi_citra_se(thresh, SED)
```



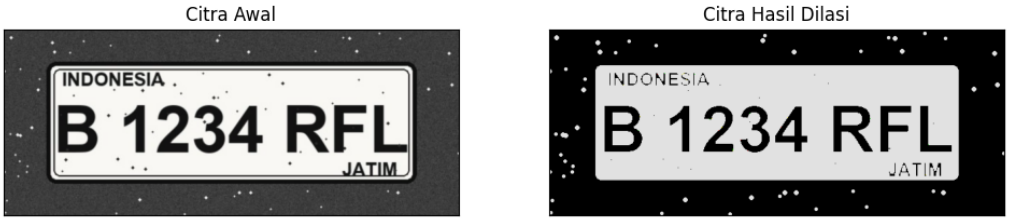
Gambar 2. Hasil dilasi dengan SED cross 3 x 3

SE cross 3 x 3 hanya menambah ketebalan ke arah vertikal dan horizontal, sehingga sudut objek tidak ikut membesar dan hasilnya lebih tipis dibanding SE persegi berukuran sama. Jumlah piksel objek pada hasil cross 3 x 3 adalah 70.906 piksel, lebih kecil dibanding SE persegi 5 x 5 karena jumlah piksel yang tercakup SE juga lebih sedikit.

Langkah 4. Dilasi dengan Library Morphology OpenCV

Proses yang sama diulang memakai fungsi cv2.dilate dengan kernel persegi 3 x 3 bertipe uint16 seperti pada modul.

```
img = cv2.imread(path('plat nomer.jpg'))
ret, thresh = cv2.threshold(img, 127, 225, cv2.THRESH_BINARY)
kernel = np.ones((3, 3), np.uint16)
dilasi = cv2.dilate(thresh, kernel, iterations=1)
```



Gambar 3. Hasil dilasi memakai cv2.dilate dengan kernel 3 x 3

Hasil cv2.dilate setara dengan fungsi manual. Perbandingan pada citra biner yang sama menunjukkan selisih 28 piksel, dengan versi manual menghasilkan 72.052 piksel objek dan versi library 72.080 piksel. Selisih hanya muncul di baris dan kolom terluar citra, karena loop manual berhenti di p - constant1 sehingga tepi citra tidak diproses, sedangkan OpenCV memproses seluruh piksel dengan border replication. Secara komputasi, versi library jauh lebih cepat karena dijalankan sebagai kode C yang teroptimasi.

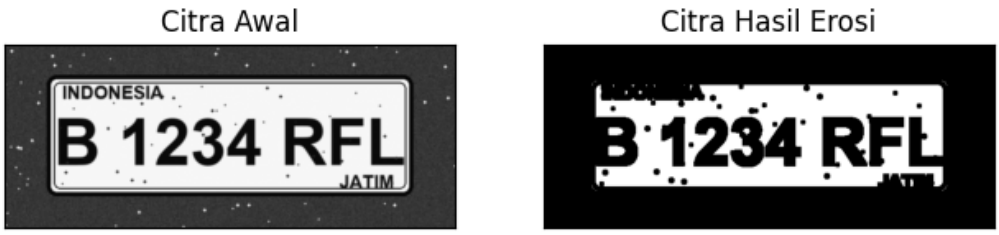
Langkah 5. Erosi tanpa Library OpenCV

Erosi dihitung manual dengan cara yang sama, tetapi nilai yang diambil adalah nilai minimum dari hasil perkalian jendela citra dengan SE. Nilai minimum dipakai karena semua piksel di dalam jendela harus bernilai objek agar piksel pusat dipertahankan.

```
def erosi_citra(F, k):
    m, n = F.shape
    SE = np.ones((k, k), dtype=np.uint8)       # structuring element persegi
    constant = (k - 1) // 2
```

```
    imgE = np.zeros((m, n), dtype=np.uint8)
    for i in range(constant, m - constant):
        for j in range(constant, n - constant):
            temp = F[i-constant:i+constant+1, j-constant:j+constant+1]
            product = temp * SE
            imgE[i, j] = np.min(product)
    return imgE

img_e = cv2.imread(path('plat nomer.jpg'), 0)
ret1, thresh1 = cv2.threshold(img_e, 175, 225, cv2.THRESH_BINARY)
imgE = erosi_citra(thresh1, 5)
```



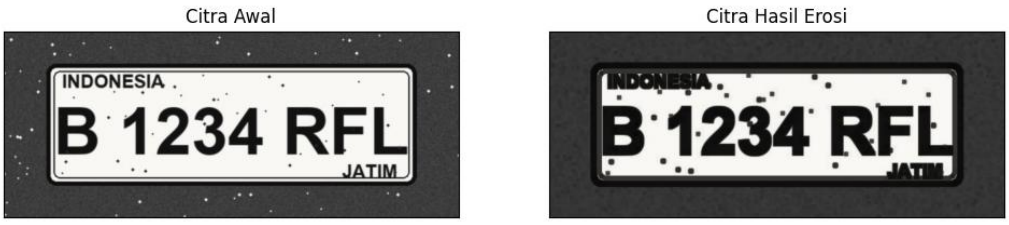
Gambar 4. Hasil erosi manual dengan SE persegi 5 x 5

Jumlah piksel objek turun dari 62.267 menjadi 45.754 piksel, atau berkurang 16.513 piksel. Goresan huruf dan garis tepi plat menjadi lebih tipis, sedangkan bintik dan garis yang ukurannya lebih kecil dari SE hilang seluruhnya. Ambang 175 pada langkah ini lebih tinggi daripada langkah sebelumnya, sehingga citra biner yang dierosi lebih bersih.

Langkah 6. Erosi dengan Library Morphology OpenCV

Erosi diulang memakai cv2.erode dengan kernel persegi 5 x 5. Sesuai modul, citra dibaca berwarna sehingga operasi bekerja pada tiga kanal sekaligus.

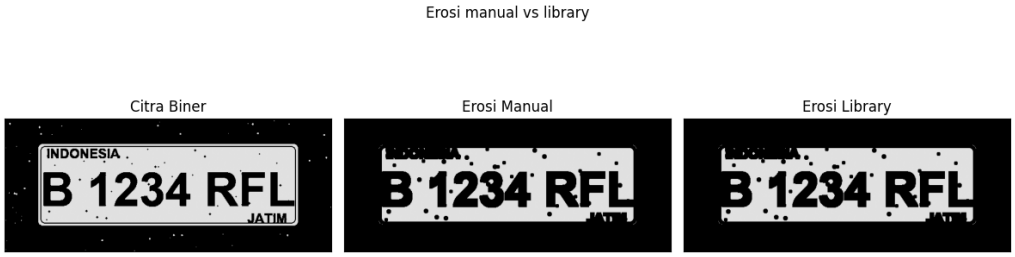
```
img = cv2.imread(path('plat nomer.jpg'))
kernel = np.ones((5, 5), np.uint8)
erosion = cv2.erode(img, kernel, iterations=1)
```



Gambar 5. Hasil erosi memakai cv2.erode dengan kernel 5 x 5

Pada citra grayscale yang sama, hasil cv2.erode identik dengan fungsi manual, dengan selisih 0 piksel. Ketika citra berwarna yang dierosi, ketiga kanal diproses terpisah dan hasilnya tetap tiga kanal. Erosi berulang dengan iterations lebih dari

satu setara dengan erosi memakai SE yang lebih besar, sehingga objek dapat hilang bila jumlah iterasi berlebihan.

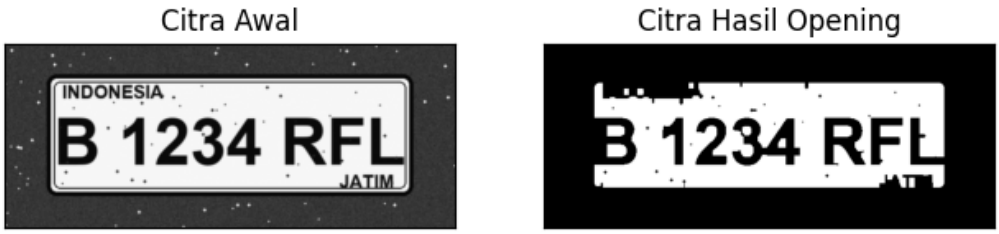


Gambar 6. Perbandingan erosi manual dan erosi library pada citra biner

Langkah 7. Operasi Opening

Opening dihitung dengan menjalankan erosi terlebih dahulu, lalu hasilnya didilasi memakai SE yang sama. Versi tanpa library memakai k = 7, sedangkan versi library memakai kernel 3 x 3 sesuai modul.

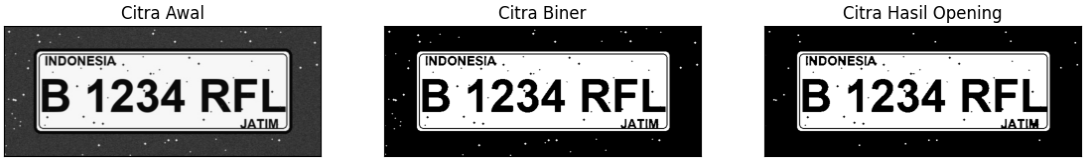
```
img_o = cv2.imread(path('plat nomer.jpg'), 0)
reto, thresho = cv2.threshold(img_o, 127, 225, cv2.THRESH_BINARY)
k = 7
imgO = dilasi_citra(erosi_citra(thresho, k), k)
```



Gambar 7. Hasil opening tanpa library dengan SE 7 x 7

Jumlah piksel objek turun dari 63.326 menjadi 55.195 piksel. Penurunan ini jauh lebih kecil dibanding erosi dengan SE yang sama, yang menghasilkan 41.634 piksel. Selisih 13.561 piksel adalah bagian objek yang dikembalikan oleh dilasi, sehingga yang benar-benar hilang hanya objek yang tidak mampu memenuhi area SE saat erosi.

```
img = cv2.imread(path('plat nomer.jpg'), 0)
ret, thresh = cv2.threshold(img, 127, 225, cv2.THRESH_BINARY)
kernel = np.ones((3, 3), np.uint8)
openn = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
```



Gambar 8. Hasil opening memakai cv2.morphologyEx dengan kernel 3 x 3

Opening menghapus bintik putih kecil di latar dan memutus jembatan tipis antar objek, sementara huruf dan garis tepi plat tetap utuh. Pada kernel 3 x 3 jumlah piksel objek hanya turun dari 63.326 menjadi 63.002, karena area yang dihapus memang kecil. Sifat idempotent juga terbukti: opening yang dijalankan dua kali menghasilkan citra yang sama, dengan selisih 0 piksel.

Langkah 8. Operasi Closing

Closing dihitung dengan menjalankan dilasi terlebih dahulu, lalu hasilnya dierosi. Versi tanpa library memakai k = 7 dan versi library memakai kernel 3 x 3.

```
img_c = cv2.imread(path('plat nomer.jpg'), 0)
retc, threshc = cv2.threshold(img_c, 127, 225, cv2.THRESH_BINARY)
k = 7
imgC = erosi_citra(dilasi_citra(threshc, k), k)
```



Gambar 9. Hasil closing tanpa library dengan SE 7 x 7

Jumlah piksel objek naik dari 63.326 menjadi 68.227 piksel. Kenaikan ini berasal dari lubang dan celah kecil di dalam huruf serta garis tepi plat yang sebelumnya bernilai latar dan sekarang terisi objek.

```
img = cv2.imread(path('plat nomer.jpg'), 0)
ret, thresh = cv2.threshold(img, 127, 225, cv2.THRESH_BINARY)
kernel = np.ones((3, 3), np.uint8)
closs = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)
```



Gambar 10. Hasil closing memakai cv2.morphologyEx dengan kernel 3 x 3

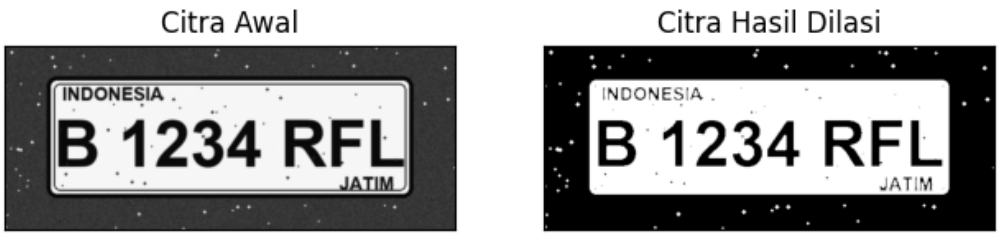
Dengan kernel 3 x 3 jumlah piksel objek naik dari 63.326 menjadi 65.970. Closing mengisi celah kecil dan menyambung garis yang retak akibat threshold, sehingga objek tampak lebih padat. Closing juga bersifat idempotent, dan bila dipasangkan dengan opening memakai SE yang sama, keduanya menjadi filter yang saling melengkapi: opening membersihkan latar, closing membersihkan objek.

Langkah 9. Dilasi dengan Structuring Element Cross

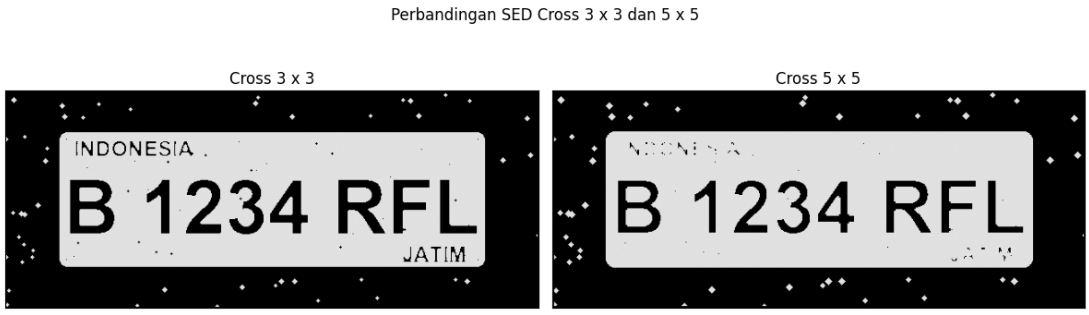
SE cross didefinisikan sebagai matriks yang hanya bernilai 1 pada baris tengah dan kolom tengah. Dilasi dijalankan untuk ukuran 3 x 3 dan 5 x 5.

```
# Cross 3 x 3
SED = np.array([[0, 1, 0],
                [1, 1, 1],
                [0, 1, 0]], dtype=np.uint8)
imgD3 = dilasi_citra_se(thresh, SED)

# Cross 5 x 5
SED5 = np.array([[0, 0, 1, 0, 0],
                 [0, 0, 1, 0, 0],
                 [1, 1, 1, 1, 1],
                 [0, 0, 1, 0, 0],
                 [0, 0, 1, 0, 0]], dtype=np.uint8)
imgD5 = dilasi_citra_se(thresh, SED5)
```



Gambar 11. Hasil dilasi dengan SED cross 3 x 3



Gambar 12. Perbandingan hasil dilasi SED cross 3 x 3 dan 5 x 5

Cross 3 x 3 menambah ketebalan objek tepat satu piksel ke atas, bawah, kiri, dan kanan, dengan jumlah piksel objek 70.906. Cross 5 x 5 menambah dua piksel ke

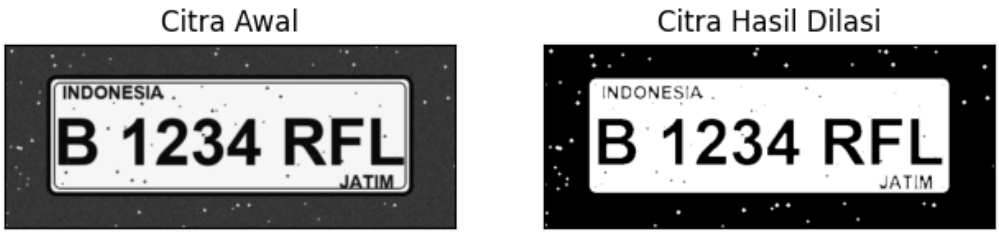
tiap arah sehingga jumlah piksel objek menjadi 75.765, huruf tampak jauh lebih tebal, dan huruf yang berdekatan mulai menyatu. Perbedaan dengan SE persegi terlihat pada sudut: SE persegi ikut mengisi diagonal sehingga sudut objek membulat dan melebar, sedangkan SE cross mempertahankan sudut asli objek.

Langkah 10. Dilasi dengan Structuring Element Circular

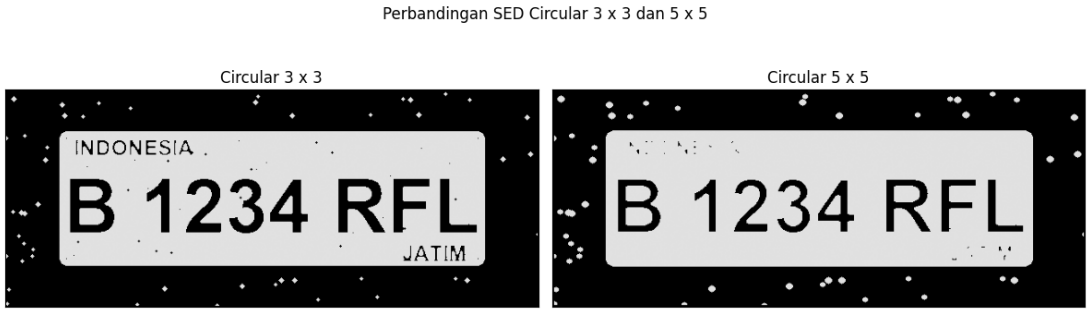
SE lingkaran dibangkitkan dengan cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (w, w)). Bentuk ini memberi penebalan yang merata ke segala arah dengan profil membulat.

```
SE_c3 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
imgD_c3 = dilasi_citra_se(thresh, SE_c3)

SE_c5 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
imgD_c5 = dilasi_citra_se(thresh, SE_c5)
```



Gambar 13. Hasil dilasi dengan SED circular 3 x 3



Gambar 14. Perbandingan hasil dilasi SED circular 3 x 3 dan 5 x 5

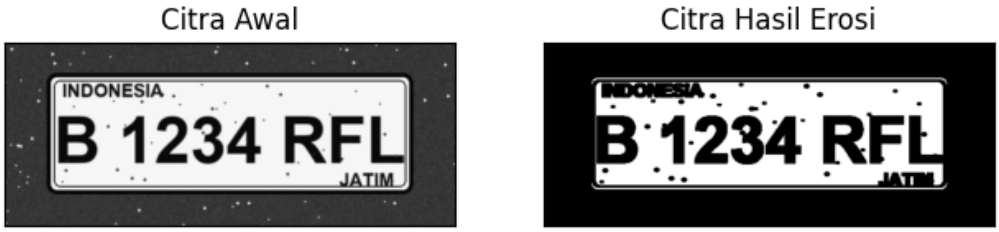
SE lingkaran 3 x 3 yang dibentuk OpenCV berupa cross dengan tambahan titik diagonal, yaitu [[0,1,0],[1,1,1],[0,1,0]], sehingga penebalannya hampir sama dengan cross namun sudut objek sedikit terisi. SE lingkaran 5 x 5 memberi penebalan membulat tanpa sudut tajam, sehingga bentuk objek lebih halus dibanding SE persegi berukuran sama. SE lingkaran cocok untuk objek yang bentuk aslinya membulat, misalnya karakter hasil pemindaian atau objek pada citra medis.

Langkah 11. Erosi dengan Structuring Element Rectangle

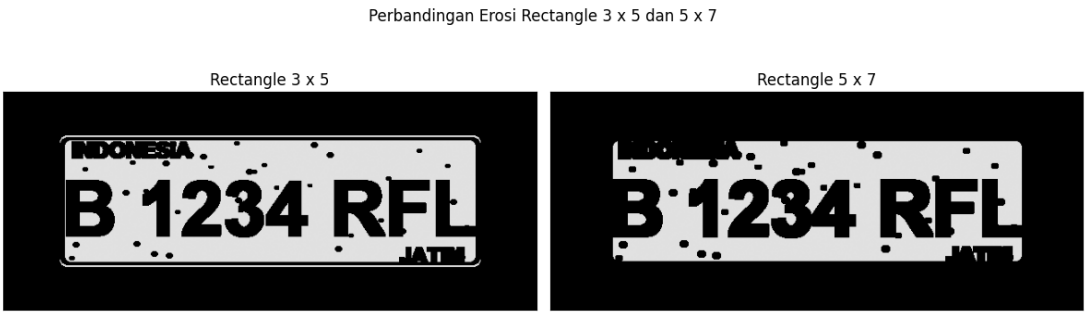
SE persegi panjang dibuat dengan cv2.getStructuringElement(cv2.MORPH_RECT, (p, l)). OpenCV menerima ukuran dalam urutan lebar dan tinggi, sehingga matriks SE berukuran l baris dan p kolom. Pemanggilan p = 5 dan l = 3 menghasilkan SE 3 x 5, dan p = 7 dengan l = 5 menghasilkan SE 5 x 7, sesuai judul langkah pada modul.

```
def erosi_citra_rect(F, p, l):
    m, n = F.shape
    SE = cv2.getStructuringElement(cv2.MORPH_RECT, (p, l))   # SE persegi
panjang
    constant1 = (l - 1) // 2
    constant2 = (p - 1) // 2
    imgE = np.zeros((m, n), dtype=np.uint8)
    for i in range(constant1, m - constant1):
        for j in range(constant2, n - constant2):
            temp = F[i-constant1:i+constant1+1, j-constant2:j+constant2+1]
            product = temp * SE
            imgE[i, j] = np.min(product)
    return imgE

imgE35 = erosi_citra_rect(thresh1, 5, 3)   # SE 3 x 5
imgE57 = erosi_citra_rect(thresh1, 7, 5)   # SE 5 x 7
```



Gambar 15. Hasil erosi dengan SE rectangle 3 x 5



Gambar 16. Perbandingan hasil erosi SE rectangle 3 x 5 dan 5 x 7

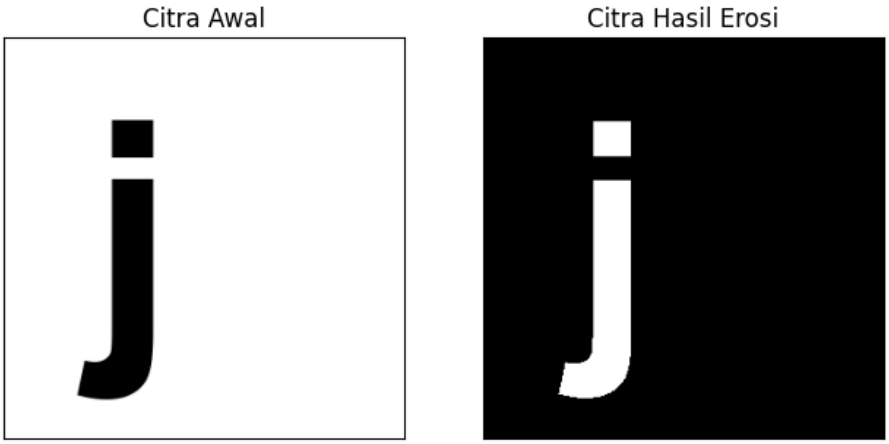
Erosi dengan SE 3 x 5 menghasilkan 51.485 piksel objek, sedangkan SE 5 x 7 menghasilkan 44.322 piksel. SE persegi panjang bersifat tidak simetris, sehingga penipisan lebih banyak terjadi pada satu arah. SE 5 x 7 menipiskan objek lebih agresif dan menghilangkan lebih banyak detail kecil, bahkan garis tepi plat yang

tipis dapat terputus. SE jenis ini berguna bila arah struktur objek sudah diketahui, misalnya untuk mempertahankan garis vertikal pada tabel atau kolom teks.

Langkah 12. Erosi dengan Structuring Element Line Vertikal

Langkah ini memakai citra j.png. SE garis vertikal dibuat dengan cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (1, k)), yaitu matriks berukuran k baris dan satu kolom.

```
def erosi_citra_line(F, k):
    m, n = F.shape
    SE = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (1, k))
    constant = (k - 1) // 2
    imgE = np.zeros((m, n), dtype=np.uint8)
    for i in range(constant, m - constant):
        for j in range(constant, n - constant):
            temp = F[i-constant:i+constant+1, j-constant:j+constant+1]
            product = temp * SE
            imgE[i, j] = np.min(product)
    return imgE
img_j = cv2.imread(path('j.png'), 0)
retj, thresh_j = cv2.threshold(img_j, 127, 225, cv2.THRESH_BINARY_INV)
imgE_l3 = erosi_citra_line(thresh_j, 3)
imgE_l5 = erosi_citra_line(thresh_j, 5)
```



Gambar 17. Hasil erosi dengan SE line vertikal 3 pada citra j.png



Gambar 18. Perbandingan hasil erosi SE line vertikal 3 dan 5

Citra j.png dibaca lalu di-threshold dengan THRESH_BINARY_INV agar huruf gelap pada latar terang berubah menjadi objek putih, sesuai asumsi operasi morfologi yang bekerja pada objek bernilai 1. SE garis vertikal hanya memeriksa tetangga atas dan bawah, sehingga erosi memotong bagian huruf yang tipis secara vertikal, sedangkan goresan vertikal yang panjang tetap bertahan. SE vertikal 5 lebih agresif daripada SE 3: bagian horizontal huruf j hilang lebih cepat dan hanya batang vertikal yang tersisa. Hasil ini menunjukkan bahwa bentuk SE menentukan arah informasi yang dipertahankan.

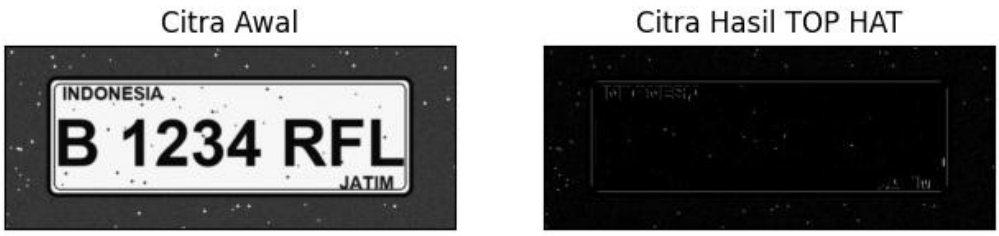
Langkah 13. Transformasi Top-hat

Top-hat dihitung dengan cv2.morphologyEx dan flag MORPH_TOPHAT memakai kernel persegi 3 x 3. Potongan kode pada modul memakai variabel input_image yang tidak didefinisikan, sehingga pada notebook variabel tersebut diganti dengan img agar kode dapat dijalankan.

```
filterSize = (3, 3)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, filterSize)

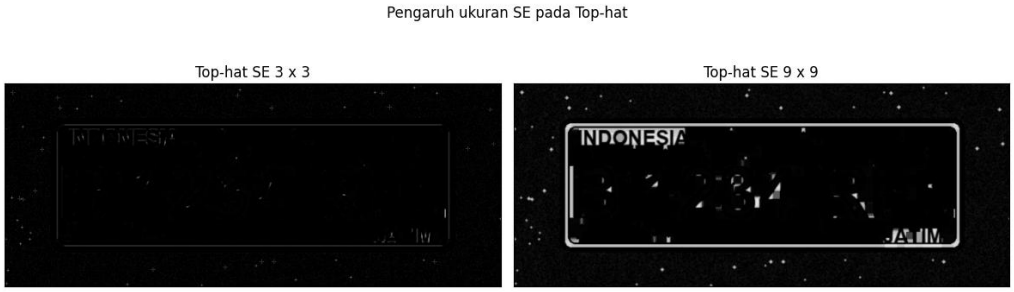
img = cv2.imread(path('plat nomer.jpg'))
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

tophat_img = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
```



Gambar 19. Hasil transformasi top-hat dengan SE 3 x 3

Rata-rata intensitas citra asli 124,91 dan rata-rata intensitas hasil top-hat hanya 2,52. Nilai ini jauh lebih rendah karena sebagian besar area yang seragam menjadi nol, sehingga citra hasil top-hat bersifat sparse: hanya detail kecil yang tersisa. Pada citra plat nomor, yang tersisa adalah tepi terang dan bintik terang, sedangkan area putih besar pada dasar plat hilang karena opening dengan SE 3 x 3 tidak mengubahnya.



Gambar 20. Pengaruh ukuran SE pada hasil top-hat

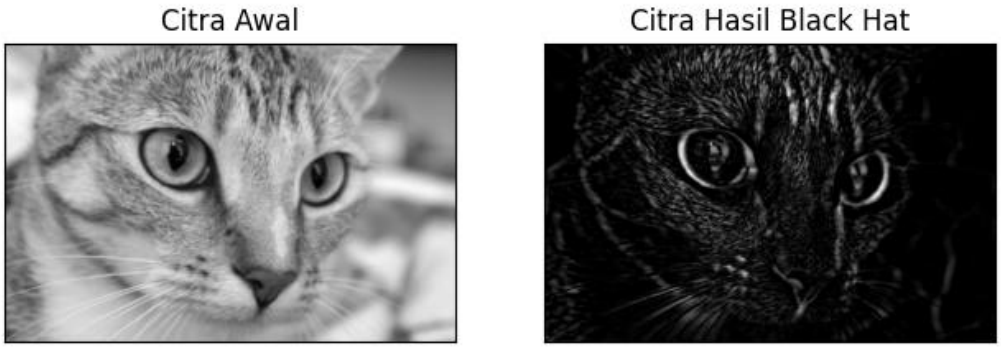
Semakin besar SE, semakin banyak objek yang dianggap kecil dan ikut tersisa pada hasil top-hat. Top-hat sering dipakai sebagai praproses sebelum threshold pada citra dengan pencahayaan tidak merata, misalnya pemindaian dokumen.

Langkah 14. Transformasi Bottom-hat

Bottom-hat dihitung dengan flag MORPH_BLACKHAT. Sesuai modul, langkah ini memakai citra kitten01.jpg dan kernel persegi panjang 13 x 5.

```
img = cv2.imread(path('kitten01.jpg'))
img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

Kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (13, 5))
blackhat = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, Kernel)
```



Gambar 21. Hasil transformasi bottom-hat dengan kernel 13 x 5

Rata-rata intensitas citra asli 116,86 dan rata-rata intensitas hasil black-hat 7,24. Black-hat menyisakan fitur gelap berukuran kecil, seperti mata, hidung, dan garis bulu pada citra kucing, sedangkan area gelap yang besar tidak tersisa karena closing dengan SE 13 x 5 sudah menutupnya. Kernel 13 x 5 bersifat asimetris, sehingga struktur gelap yang memanjang ke samping lebih banyak tersisa dibanding yang memanjang ke atas. Top-hat dan bottom-hat merupakan pasangan:

top-hat menonjolkan objek terang kecil di latar gelap, bottom-hat menonjolkan objek gelap kecil di latar terang.

Langkah 15. Skeleton

Skeleton dihitung dengan skimage.morphology.skeletonize setelah citra dibinerkan memakai filters.threshold_triangle. Nilai boolean diubah menjadi 0 dan 1 terlebih dahulu agar tipe data sesuai dengan fungsi skeletonize.

```
from skimage import filters
from skimage.morphology import skeletonize

img = cv2.imread(path('lily.jpg'), 0)
binary = img > filters.threshold_triangle(img)

binary_cp = binary.copy()
binary_cp[binary_cp == True] = 1
binary_cp[binary_cp == False] = 0

skeleton = skeletonize(binary_cp)
```



Gambar 22. Skeleton citra lily dengan threshold triangle

Nilai threshold triangle yang diperoleh adalah 11, sehingga hampir seluruh bagian terang bunga menjadi objek, dengan 95.885 piksel objek. Skeleton yang terbentuk hanya 1.829 piksel, atau 1,91 persen dari piksel objek. Skeleton mempertahankan struktur utama bunga, yaitu kelopak dan putik, dengan ketebalan satu piksel. Sifat skeleton terlihat pada hasil: tipis, terhubung mengikuti topologi asli, berada di tengah objek, dan stabil terhadap pengikisan lanjutan. Perbandingan jumlah piksel ini menunjukkan efisiensi representasi bentuk, dan panjang skeleton dapat dipakai untuk menghitung intensitas rekahan $I = \Sigma\, li / V$ seperti pada modul.

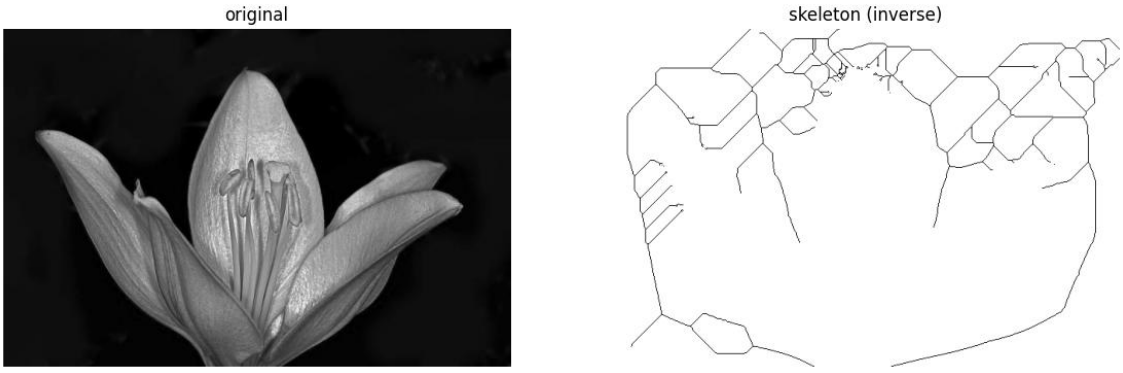
Langkah 16. Skeleton Inverse

Skeleton kedua memakai citra yang dibaca dengan skimage.io dan dibinerkan dengan operator kurang dari, sehingga bagian gelap yang menjadi objek. Colormap gray_r dipakai agar nilai skeleton tampil sebagai garis gelap pada latar terang.

```
from skimage import io
from skimage import filters
from skimage.morphology import skeletonize

img = io.imread(path('lily.jpg'))[..., 0]

# bagian gelap dijadikan True
binary = img < filters.threshold_triangle(img)
skeleton = skeletonize(binary)
```



Gambar 23. Skeleton inverse dengan operator kurang dari

Dengan operator kurang dari, piksel gelap menjadi objek, sehingga jumlah piksel objek menjadi 175.725 dan skeleton yang terbentuk 4.340 piksel. Skeleton mengikuti bagian gelap citra, yaitu bayangan dan kontur dalam kelopak bunga, bukan lagi bentuk luar bunga. Perbedaan hasil dengan langkah sebelumnya menunjukkan bahwa skeleton sangat bergantung pada hasil segmentasi: perubahan aturan threshold membalik objek yang dianalisis dan menghasilkan kerangka yang sama sekali berbeda.

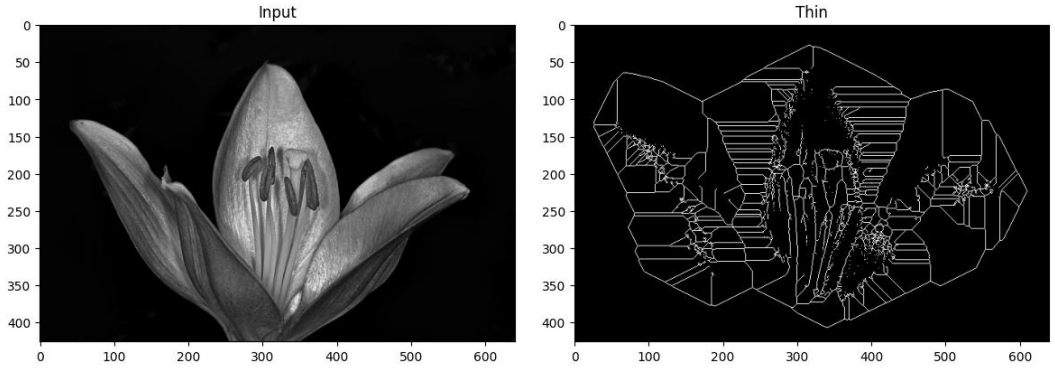
Langkah 17. Thickening dan Thinning

Scikit-image menyediakan morphology.thin untuk menipiskan objek. Penebalan dihitung dengan menerapkan thin pada komplemen citra, lalu menggabungkan hasilnya dengan objek asli, sesuai persamaan $A \oplus B = A \cup (Ac \otimes B)$.

```
from skimage import img_as_float
from skimage import io, color, morphology
```

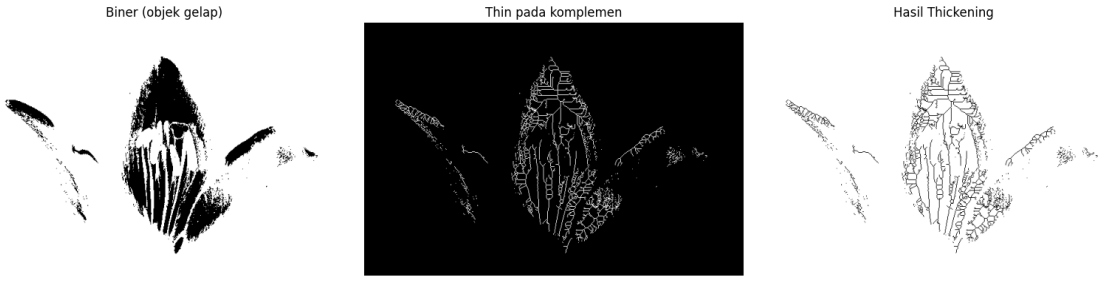
```
image = img_as_float(color.rgb2gray(io.imread(path('lily.jpg'))))
image_binary = image < 0.5
out_skeletonize = morphology.skeletonize(image_binary)
out_thin = morphology.thin(image_binary)

# thickening: thin pada komplemen, lalu digabung dengan objek asli
tebal = morphology.thin(~image_binary)
hasil_tebal = image_binary | ~tebal
```



Gambar 24. Hasil thinning pada citra biner lily

Citra biner dengan aturan image < 0,5 memiliki 246.422 piksel objek. Hasil thin sebanyak 13.017 piksel dan hasil skeletonize 12.993 piksel. Keduanya menghasilkan objek setebal satu piksel, tetapi thin tidak menjamin kerangka berada tepat di tengah objek, sehingga jumlah pikselnya sedikit berbeda dengan skeletonize.



Gambar 25. Proses dan hasil thickening

Setelah thickening, jumlah piksel objek bertambah dari 246.422 menjadi 266.508. Objek bertambah tebal pada bagian yang sebelumnya tipis atau berlubang. Pertambahan ini kebalikan dari erosi dan thinning yang selalu mengurangi jumlah piksel objek.

Rekapitulasi Hasil

Tabel 1. Perubahan jumlah piksel objek pada citra plat nomor

Operasi	Piksel objek sebelum	Piksel objek sesudah	Perubahan
Dilasi manual, SE persegi 5 x 5	63.367	77.866	+14.499
Dilasi SED cross 3 x 3	63.367	70.906	+7.539
Dilasi SED cross 5 x 5	63.367	75.765	+12.398
Erosi manual, SE persegi 5 x 5	62.267	45.754	-16.513
Opening manual, SE 7 x 7	63.326	55.195	-8.131
Opening library, kernel 3 x 3	63.326	63.002	-324
Closing manual, SE 7 x 7	63.326	68.227	+4.901
Closing library, kernel 3 x 3	63.326	65.970	+2.644
Erosi rectangle 3 x 5	62.267	51.485	-10.782
Erosi rectangle 5 x 7	62.267	44.322	-17.945

Tabel di atas menunjukkan pola yang konsisten: erosi selalu mengurangi jumlah piksel objek, dilasi selalu menambah, opening mengurangi lebih sedikit dibanding erosi dengan SE yang sama, dan closing menambah jumlah piksel objek. Besarnya perubahan ditentukan oleh ukuran SE, bukan hanya oleh jenis operasinya.

E. TUGAS PRAKTIKUM: MORFOLOGI PADA CITRA KTP

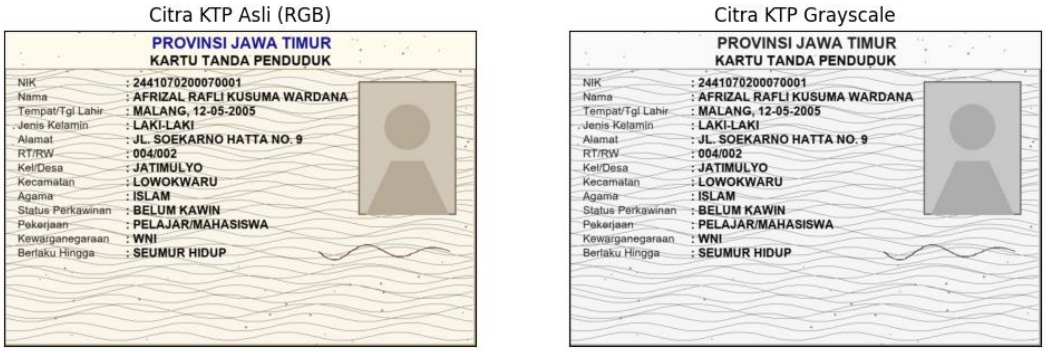
Tugas akhir modul meminta seluruh teknik morfologi diterapkan pada citra KTP dari dataset Modul 7 agar teks pada KTP terbaca lebih baik. Citra KTP memiliki ciri khas: latar terang dengan pola garis halus (guilloche), teks gelap berukuran kecil, dan elemen foto yang mengandung gradasi. Tahapan yang dilakukan adalah segmentasi dengan threshold Otsu, opening untuk menghapus pola latar, closing untuk menyambung goresan huruf, top-hat dan bottom-hat untuk menonjolkan detail kecil, serta skeleton untuk memeriksa ketebalan goresan huruf.

1. Membaca Citra dan Segmentasi

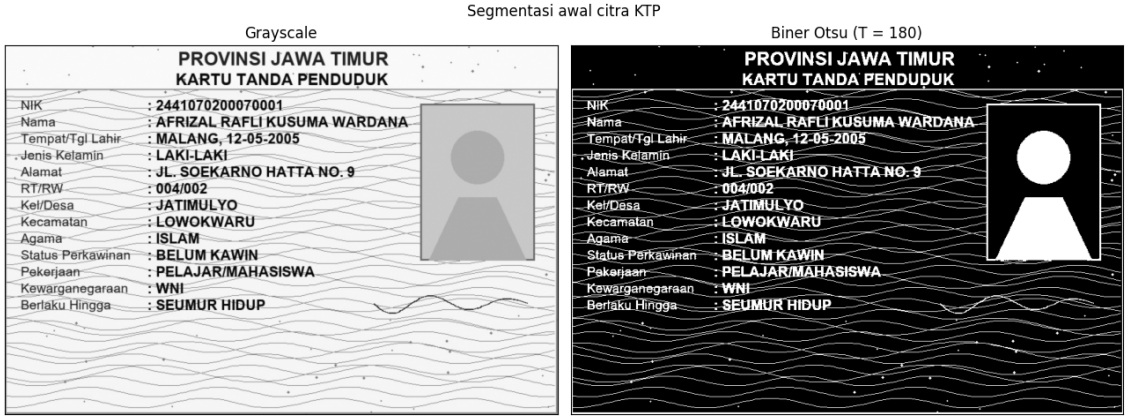
Citra KTP berukuran 900 x 600 piksel dibaca berwarna, lalu diubah ke grayscale. Segmentasi memakai threshold Otsu dengan operator THRESH_BINARY_INV agar teks yang gelap menjadi objek putih yang dapat diproses morfologi.

```
ktp_bgr = cv2.imread(path('ktp.jpg'))
ktp = cv2.cvtColor(ktp_bgr, cv2.COLOR_BGR2GRAY)

nilai_otsu, ktp_biner = cv2.threshold(
    ktp, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```



Gambar 26. Citra KTP asli dan hasil konversi grayscale



Gambar 27. Hasil segmentasi Otsu pada citra KTP

Nilai threshold Otsu yang diperoleh adalah 180. Hasil segmentasi memiliki 94.877 piksel objek atau 17,57 persen dari luas citra, dengan 103 komponen terhubung. Angka ini jauh lebih besar daripada luas teks sebenarnya karena pola garis guilloche pada latar juga ikut tertangkap threshold.

2. Opening dan Closing

Opening memakai SE persegi 2 x 2 untuk menghapus garis latar setebal satu piksel, lalu closing memakai SE 2 x 2 untuk menyambung goresan huruf yang terputus.

```
kernel_open = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
ktp_open = cv2.morphologyEx(ktp_biner, cv2.MORPH_OPEN, kernel_open)

kernel_close = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
ktp_close = cv2.morphologyEx(ktp_open, cv2.MORPH_CLOSE, kernel_close)
```



Gambar 28. Hasil opening lalu closing pada citra KTP

Tabel 2. Perubahan jumlah piksel dan komponen setelah opening dan closing

Tahap	Piksel objek	Persentase (%)	Komponen terhubung
Biner Otsu	94.877	17,57	103
Setelah opening	66.016	12,23	505
Setelah opening + closing	65.867	12,20	435

Opening menghapus garis guilloche setebal satu piksel, karena erosi 2 x 2 tidak mungkin mempertahankan objek setipis itu. Persentase piksel objek turun dari 17,57 menjadi 12,23 persen. Jumlah komponen terhubung justru naik dari 103 menjadi 505: jaringan garis yang sebelumnya saling menyambung terpotong menjadi pecahan kecil, dan setiap pecahan terhitung sebagai komponen baru. Teks utama tetap utuh karena goresannya lebih tebal dari SE.

Closing 2 x 2 kemudian mengisi celah kecil pada goresan huruf yang terputus akibat threshold. Jumlah komponen turun dari 505 menjadi 435, yang berarti potongan huruf dan sisa garis yang berdekatan menyatu kembali, dan huruf tampak lebih padat tanpa bertambah tebal berlebihan.

3. Pengaruh Ukuran Kernel Opening

Untuk memastikan ukuran kernel yang tepat, opening diulang dengan kernel 3 x 3.

```
ktp_open3 = cv2.morphologyEx(
    ktp_biner, cv2.MORPH_OPEN,
    cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
```



Gambar 29. Perbandingan opening dengan kernel 2 x 2 dan 3 x 3

Opening 3 x 3 menghapus lebih banyak garis latar, tetapi mulai menggerus goresan huruf yang tipis. Persentase piksel objek turun lebih jauh, dari 12,23

menjadi 10,77 persen, dan jumlah komponen turun menjadi 432. Karena itu kernel 2 x 2 dipilih sebagai titik seimbang antara kebersihan latar dan keutuhan huruf.

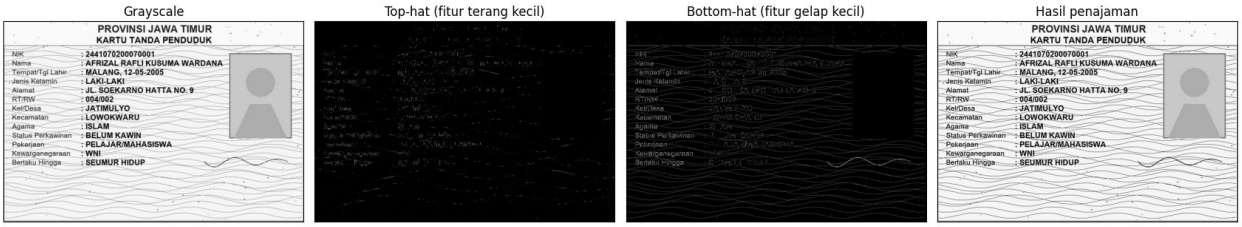
4. Top-hat dan Bottom-hat

Top-hat dan bottom-hat dihitung pada citra grayscale dengan kernel 3 x 3, lalu digabungkan menjadi operasi penajaman kontras: citra ditambah top-hat dan dikurangi bottom-hat.

```
kernel_th = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
ktp_tophat = cv2.morphologyEx(ktp, cv2.MORPH_TOPHAT, kernel_th)
ktp_blackhat = cv2.morphologyEx(ktp, cv2.MORPH_BLACKHAT, kernel_th)

ktp_enhance = cv2.add(ktp, ktp_tophat)
ktp_enhance = cv2.subtract(ktp_enhance, ktp_blackhat)
```

Top-hat, Bottom-hat, dan penajaman kontras



Gambar 30. Top-hat, bottom-hat, dan hasil penajaman kontras pada KTP

Top-hat menyisakan detail terang kecil, bottom-hat menyisakan detail gelap kecil. Kombinasi citra ditambah top-hat dan dikurangi bottom-hat menaikkan kontras lokal sehingga tepi huruf lebih tegas dibanding citra grayscale asli.

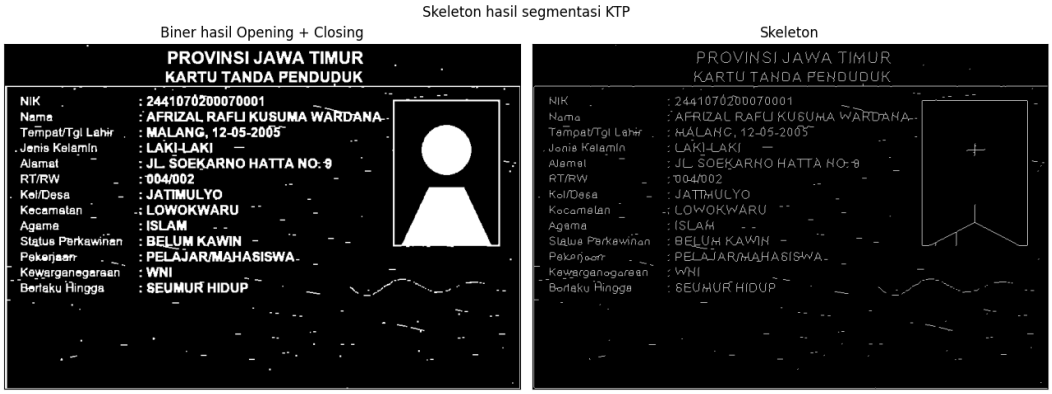
5. Skeleton dan Evaluasi Keterbacaan

Skeleton dihitung dari hasil segmentasi untuk memeriksa ketebalan goresan huruf. Keterbacaan kemudian dibandingkan memakai dua metrik sederhana, yaitu standar deviasi intensitas sebagai ukuran kontras dan varians Laplacian sebagai ukuran ketajaman tepi.

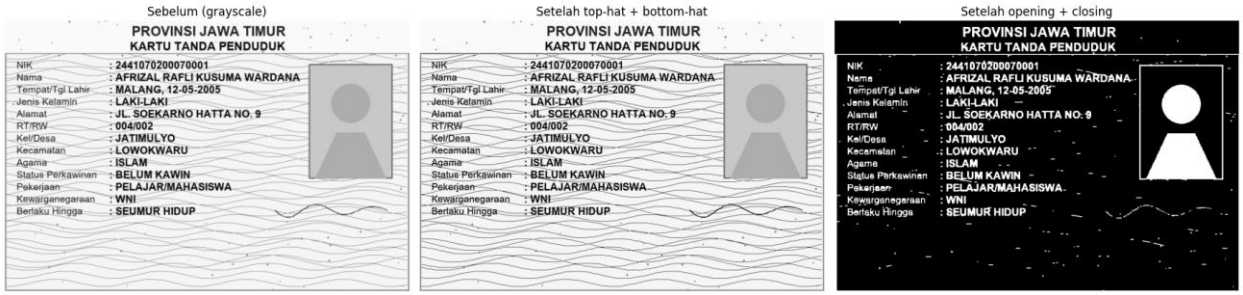
```
from skimage.morphology import skeletonize

ktp_skel = skeletonize(ktp_close > 0)

def skor_keterbacaan(citra):
    lap = cv2.Laplacian(citra, cv2.CV_64F)
    return float(citra.std()), float(lap.var())
```



Gambar 31. Skeleton dari hasil segmentasi citra KTP



Gambar 32. Perbandingan citra KTP sebelum dan sesudah rangkaian morfologi

Tabel 3. Perbandingan metrik keterbacaan citra KTP

Tahap	Standar deviasi intensitas	Varians Laplacian
Grayscale	53,46	4.097,37
Setelah penajaman	65,30	13.564,26
Biner akhir	83,45	11.059,58

Standar deviasi intensitas naik dari 53,46 pada citra grayscale menjadi 65,30 setelah penajaman dan 83,45 pada citra biner akhir. Kenaikan ini menandakan sebaran nilai piksel makin lebar, yaitu kontras makin tinggi. Varians Laplacian juga naik dari 4.097,37 menjadi 13.564,26 setelah penajaman, yang menandakan tepi objek makin tajam. Skeleton dari hasil segmentasi memiliki 14.712 piksel, jauh lebih sedikit dibanding 65.867 piksel objek, dan bentuk karakter tetap dapat dikenali meski sebagian besar piksel dihilangkan.

6. Analisis Tugas Praktikum

1. Threshold Otsu memisahkan teks dari latar KTP secara otomatis tanpa ambang manual. Operator THRESH_BINARY_INV dipakai karena latar KTP lebih terang daripada teks, sehingga teks menjadi objek putih yang dapat diproses

morfologi. Hasil segmentasi masih membawa pola garis guilloche dan bintik, sehingga jumlah piksel objek jauh lebih besar daripada luas teks sebenarnya.

2. Opening dengan SE 2 x 2 menghapus garis guilloche setebal satu piksel, karena erosi 2 x 2 tidak mungkin mempertahankan objek setipis itu. Jumlah komponen terhubung naik karena jaringan garis yang saling menyambung terpotong menjadi pecahan kecil, bukan karena noise bertambah.
3. Closing 2 x 2 mengisi celah kecil pada goresan huruf yang terputus akibat threshold. Jumlah komponen turun dari 505 menjadi 435, yang berarti potongan huruf dan sisa garis yang berdekatan menyatu kembali.
4. Ukuran kernel menentukan hasil akhir. Opening 3 x 3 menghapus lebih banyak garis latar, tetapi mulai menggerus goresan huruf yang tipis, sehingga kernel 2 x 2 lebih tepat untuk citra ini.
5. Top-hat dan bottom-hat menaikkan kontras lokal. Kombinasi citra ditambah top-hat dan dikurangi bottom-hat membuat tepi huruf lebih tegas, dengan standar deviasi intensitas naik dari 53,46 menjadi 65,30 dan varians Laplacian naik dari 4.097,37 menjadi 13.564,26.
6. Skeleton mempertahankan topologi huruf sehingga bentuk karakter tetap dapat dikenali meski sebagian besar piksel dihilangkan, dan dapat dipakai untuk mengukur ketebalan goresan huruf.
7. Rangkaian threshold Otsu, opening 2 x 2, dan closing 2 x 2 adalah kombinasi yang paling efektif untuk memperbaiki keterbacaan KTP pada praktikum ini. Hasil akhirnya lebih siap untuk tahap pengenalan karakter (OCR) dibanding citra asli.

F. KESIMPULAN

1. **Structuring element adalah inti morfologi.** Bentuk dan ukuran SE menentukan arah perubahan objek. SE persegi menebal atau menipis ke segala arah, SE cross hanya ke arah vertikal dan horizontal, SE lingkaran memberi perubahan membulat, dan SE garis vertikal hanya mempertahankan struktur vertikal.
2. **Erosi dan dilasi adalah pasangan.** Erosi mengurangi jumlah piksel objek dan menghilangkan detail yang lebih kecil dari SE. Dilasi menambah jumlah piksel objek dan menutup lubang kecil. Keduanya bersifat dual: dilasi pada objek setara dengan erosi pada latar.
3. **Opening dan closing adalah operasi turunan yang lebih terkontrol.** Opening menghapus noise eksternal tanpa mengubah ukuran objek utama. Closing menutup lubang internal dan menyambung objek yang retak. Keduanya idempotent, sehingga aman diulang.
4. **Top-hat dan bottom-hat mengekstrak detail kecil.** Top-hat menyisakan fitur terang kecil, bottom-hat menyisakan fitur gelap kecil. Keduanya berguna sebagai praproses sebelum threshold pada citra dengan pencahayaan tidak merata.
5. **Skeleton dan thickening melengkapi analisis bentuk.** Skeleton meringkas objek menjadi kerangka satu piksel dengan topologi tetap, berguna untuk mengukur panjang dan intensitas rekahan. Thickening, sebagai kebalikan thinning, menebalkan objek dengan menipiskan komplemennya.
6. **Implementasi manual dan library menghasilkan keluaran yang setara.** Selisih hanya terjadi di tepi citra karena penanganan border yang berbeda, sedangkan versi library jauh lebih cepat. Pemahaman implementasi manual tetap penting agar perilaku setiap operator dapat dijelaskan, bukan hanya dipakai.
7. **Penerapan pada KTP.** Kombinasi threshold Otsu, opening, dan closing berhasil membersihkan pola latar dan menyambung goresan huruf KTP. Top-hat dan bottom-hat menaikkan kontras lokal, sedangkan skeleton memastikan struktur huruf tetap terjaga.